Лекции

- Лекция 1: Введение в Java
- Лекция 2: Основные конструкции Java
- Лекция 3: Ветвление, Циклы и ООП
- Лекция 4: Вложенные классы, Обобщени…
- Лекция 5: Коллекции, I/O и Многопоточн…
- Лекция 6: Системы сборки, JDBC и Hiber…

Практические занятия

- Практика 1: Основы Java
- Практика 2: Основные конструкции Java
- Практика 3: Ветвление, Циклы и ООП
- Практика 4: Вложенные классы, Обобще…
  - Часть 1: Вложенные классы
    - Задание 1.1: Нестатический внутр…
    - Задание 1.2: Статический вложенн…
    - Задание 1.3: Сравнение Inner vs St…
  - Часть 2: Вложенные интерфейсы
    - Задание 2.1: Система событий
  - Часть 3: Обобщения (Generics)
    - Задание 3.1: Обобщённый стек
    - Задание 3.2: Ограниченные парам…
    - Задание 3.3: Wildcards — копирова…
  - Часть 4: Исключения
    - Задание 4.1: Базовая обработка и…
    - Задание 4.2: Собственные исключ…
    - Задание 4.3: Try-with-resources
    - Задание 4.4: Цепочка исключений
  - Часть 5: Самостоятельная работа

Практическое занятие 4: Вложенные классы, Обобщения и Исключения

Часть 1: Вложенные классы

Задание 1.1: Нестатический внутренний класс

Реализуйте класс `Library` с полями `name` и `capacity`. Внутри него объявите нестатический внутренний класс `Book` с полями `title` (String), `author` (String), `year` (int). Метод `getInfo()` должен выводить: `«Книга: [title] автора [author] ([year]) в библиотеке [Library.name]»`. Обратите внимание: `Book` обращается к полю `name` внешнего класса напрямую. Создайте экземпляр `Library`, затем экземпляр `Book` через `lib.new Book(...)`, вызовите `getInfo()`.

Ожидаемый вывод:

```
Книга: Война и мир автора Толстой Л.Н. (1869) в библиотеке Городская библиотека
```

Вопрос: Попробуйте добавить `static` к классу `Book`. Что изменится? Почему потеряется доступ к `name`?

Задание 1.2: Статический вложенный класс — Builder Pattern

Реализуйте класс `Computer` с `final` -полями: `cpu` (String), `ram` (int), `storage` (int, по умолчанию 256), `gpu` (String, по умолчанию `null`), `ssd` (boolean, по умолчанию `true`). Конструктор `private` — принимает `Builder`.

Реализуйте статический вложенный класс `Builder` с конструктором `Builder(String cpu, int ram)` и методами `storage(int)`, `gpu(String)`, `ssd(boolean)` (каждый возвращает `this`) и `build()` (возвращает новый `Computer`).

Переопределите `toString()` в формате: `Computer{CPU='...', RAM=...GB, Storage=...GB SSD/HDD, GPU='...' / 'встроенная'}`.

Создайте игровой компьютер (Core i9, 32GB, 2000GB, RTX 4080, SSD) и офисный (Core i5, 16GB, дефолтные значения) и выведите оба.

Задание 1.3: Сравнение Inner vs Static Nested

Запустите следующий код и объясните результаты:

```java
public class Memory {
    private int instanceData = 100;
    private static int staticData = 200;

    class InnerClass {
        void show() {
            System.out.println("Inner: instanceData = " + instanceData);  // ?
            System.out.println("Inner: staticData = " + staticData);      // ?
        }
    }

    static class StaticNested {
        void show() {
            // System.out.println("Static: instanceData = " + instanceData); // ?
            System.out.println("Static: staticData = " + staticData);     // ?
        }
    }

    public static void main(String[] args) {
        Memory memory = new Memory();

        // Нестатический — нужен экземпляр
        InnerClass inner = memory.new InnerClass();
        inner.show();

        // Статический — экземпляр не нужен
        StaticNested nested = new StaticNested();
        nested.show();
    }
}
```

Ответьте письменно:

1. Почему закомментированная строка вызовет ошибку компиляции?
2. В чём разница при создании экземпляров этих двух классов?

Часть 2: Вложенные интерфейсы

Задание 2.1: Система событий

Реализуйте класс `EventSystem` со следующими вложенными элементами:

- Вложенный интерфейс `EventHandler<T>` с методом `void handle(T event)`.
- Вложенный интерфейс `EventFilter<T>` с методом `boolean accept(T event)`.
- Вложенный статический класс `Event` с полями `type` (String), `data` (String), `timestamp` (long, устанавливается при создании).
- В самом `EventSystem`: список обработчиков (`List<EventHandler<Event>>`), текущий фильтр (`EventFilter<Event>`), методы `setFilter`, `addHandler`, `fire`.

В методе `fire(Event event)`: если фильтр есть и он не принимает событие — вывести `«Событие отфильтровано: ...»`; иначе — передать всем обработчикам.

В `main()`: установите фильтр, пропускающий только `type == "ERROR"`. Добавьте обработчик, выводящий событие. Запустите 4 события (INFO, ERROR, DEBUG, ERROR).

Часть 3: Обобщения (Generics)

Задание 3.1: Обобщённый стек

Реализуйте обобщённый стек `Stack<T>` (LIFO) на основе массива `Object[]`:

- Начальная ёмкость 10; при переполнении массив расширяется вдвое.
- `push(T element)` — добавляет на вершину.
- `pop()` — удаляет и возвращает вершину; бросает `EmptyStackException` если пуст.
- `peek()` — возвращает вершину без удаления; бросает `EmptyStackException` если пуст.
- `isEmpty()`, `size()`, `toString()`.

Продемонстрируйте работу: создайте `Stack<String>`, добавьте три строки, вызовите `peek()` и `pop()`. Создайте `Stack<Integer>`, заполните числами 10–50, извлеките все элементы в цикле.

Задание 3.2: Ограниченные параметры типа

Реализуйте утилитарный класс `NumberUtils` со следующими статическими методами:

- `sum(List<? extends Number> numbers)` → `double` — сумма элементов.
- `average(List<? extends Number> numbers)` → `double` — среднее; бросает `IllegalArgumentException` для пустого списка.
- `<T extends Comparable<T>> T findMax(List<T> list)` — максимальный элемент; бросает `IllegalArgumentException` для `null` или пустого списка.
- `<T extends Comparable<T>> T findMin(List<T> list)` — аналогично.
- `fillWithIntegers(List<? super Integer> list, int n)` — добавляет в список числа от 1 до n.

Протестируйте: сумма `[1,2,3,4,5]`, сумма `[1.5,2.5,3.5]`, среднее, максимум строк `["яблоко","апельсин","банан"]`, `fillWithIntegers` в `List<Number>`.

Задание 3.3: Wildcards — копирование коллекций

Реализуйте класс `WildcardDemo` с тремя статическими методами, правильно выбрав wildcards:

- `printAll(List<?> list)` — выводит все элементы (принимает список любого типа).
- `sumNumbers(List<? extends Number> list)` → `double` — сумма через `Number::doubleValue`.
- `addDefaults(List<? super String> list)` — добавляет `"default1"` и `"default2"`.

Объясните в комментарии: почему `List<Integer>` не присваивается переменной `List<Number>`? Как это обойти с помощью wildcard? Продемонстрируйте корректную копию из `List<Integer>` в `List<Number>`.

Часть 4: Исключения

Задание 4.1: Базовая обработка исключений

Напишите программу `ExceptionBasics` с тремя блоками:

1. Деление: для массива делителей `{2, 0, 5, 0, 1}` вычислите `100 / divisor`, обработав `ArithmeticException`.
2. Массив: для индексов `{0, 1, 5, 2, -1}` обратитесь к массиву `{10, 20, 30}`, обработав `ArrayIndexOutOfBoundsException`.
3. Парсинг: для строк `{"42", "abc", "100", "3.14", "-7"}` вызовите `Integer.parseInt`, обработав `NumberFormatException`.

Для каждого случая распечатайте результат или сообщение об ошибке.

Задание 4.2: Собственные исключения

Создайте систему исключений для банковского приложения:

```java
// 1. Базовое checked исключение для банковских операций
public class BankException extends Exception {
    private String operationId;

    public BankException(String message, String operationId) {
        super(message);
        this.operationId = operationId;
    }

    public String getOperationId() { return operationId; }
}

// 2. Исключение недостаточных средств
public class InsufficientFundsException extends BankException {
    private double required;
    private double available;

    public InsufficientFundsException(double required, double available, String operationId) {
        super(String.format("Недостаточно средств. Требуется: %.2f, доступно: %.2f",
              required, available), operationId);
        this.required = required;
        this.available = available;
    }

    public double getRequired() { return required; }
    public double getAvailable() { return available; }
}

// 3. Unchecked исключение — неверный аргумент
public class InvalidAmountException extends RuntimeException {
    public InvalidAmountException(double amount) {
        super("Сумма должна быть положительной, получено: " + amount);
    }
}

// 4. Класс счёта
public class BankAccount {
    private String accountId;
    private double balance;

    public BankAccount(String accountId, double initialBalance) {
        if (initialBalance < 0) {
            throw new InvalidAmountException(initialBalance);
        }
        this.accountId = accountId;
        this.balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new InvalidAmountException(amount);
        }
        balance += amount;
        System.out.printf("Пополнение на %.2f. Баланс: %.2f%n", amount, balance);
    }

    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount <= 0) {
            throw new InvalidAmountException(amount);
        }
        if (amount > balance) {
            throw new InsufficientFundsException(amount, balance, "WD-" + accountId);
        }
        balance -= amount;
        System.out.printf("Снятие %.2f. Баланс: %.2f%n", amount, balance);
    }

    public double getBalance() { return balance; }
    public String getAccountId() { return accountId; }
}

// 5. Тест
public class BankTest {
    public static void main(String[] args) {
        BankAccount account = new BankAccount("ACC-001", 1000.0);

        // Тест нормального пополнения и снятия
        account.deposit(500.0);
        try {
            account.withdraw(200.0);
        } catch (InsufficientFundsException e) {
            System.out.println("Ошибка: " + e.getMessage());
        }

        // Тест недостаточных средств
        try {
            account.withdraw(2000.0); // Должно бросить InsufficientFundsException
        } catch (InsufficientFundsException e) {
            System.out.println("Ошибка: " + e.getMessage());
            System.out.printf("Не хватает: %.2f%n", e.getRequired() - e.getAvailable());
        }

        // Тест невалидной суммы (unchecked)
        try {
            account.deposit(-100); // Должно бросить InvalidAmountException
        } catch (InvalidAmountException e) {
            System.out.println("Ошибка: " + e.getMessage());
        }

        // Тест невалидного начального баланса
        try {
            BankAccount badAccount = new BankAccount("BAD-001", -500);
        } catch (InvalidAmountException e) {
            System.out.println("Нельзя создать счёт: " + e.getMessage());
        }
    }
}
```

Задание 4.3: Try-with-resources

Изучите класс `FileLogger`, реализующий `AutoCloseable`. Запустите программу и убедитесь, что `close()` вызывается автоматически. Объясните: (1) что выводится при `close()`? (2) почему `transient` не применяется здесь, а `flush()` нужен?

```java
import java.io.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class FileLogger implements AutoCloseable {
    private PrintWriter writer;
    private String filename;
    private int entriesWritten = 0;

    public FileLogger(String filename) throws IOException {
        this.filename = filename;
        this.writer = new PrintWriter(new FileWriter(filename, true));
        System.out.println("Логгер открыт: " + filename);
    }

    public void log(String level, String message) {
        String timestamp = LocalDateTime.now()
            .format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));
        writer.printf("[%s] [%s] %s%n", timestamp, level, message);
        writer.flush(); // Сбрасываем буфер
        entriesWritten++;
    }

    public void info(String message) {
        log("INFO", message);
    }

    public void error(String message) {
        log("ERROR", message);
    }

    public void warning(String message) {
        log("WARNING", message);
    }

    @Override
    public void close() {
        writer.close();
        System.out.println("Логгер закрыт: " + filename + " (записей: " + entriesWritten + ")");
    }

    public static void main(String[] args) {
        try (FileLogger logger = new FileLogger("app.log")) {
            logger.info("Приложение запущено");
            logger.info("Инициализация завершена");
            logger.warning("Конфигурация не найдена, используются значения по умолчанию");

            // Симулируем ошибку
            try {
                int result = 10 / 0;
            } catch (ArithmeticException e) {
                logger.error("Ошибка: " + e.getMessage());
            }

            logger.info("Работа завершена");
        } catch (IOException e) {
            System.out.println("Не удалось открыть файл лога: " + e.getMessage());
        }
        // logger.close() вызвался автоматически!

        // Прочитаем записанный файл
        System.out.println("\n--- Содержимое app.log ---");
        try (BufferedReader reader = new BufferedReader(new FileReader("app.log"))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Ошибка чтения: " + e.getMessage());
        }
    }
}
```

Задание 4.4: Цепочка исключений

Изучите и запустите трёхуровневую систему. Объясните: (1) что выведет `e.getCause().getMessage()`? (2) для чего используется цепочка исключений? (3) какова разница между `getMessage()` и `getCause().getMessage()`?

```java
// Уровень 1: Работа с базой данных (низкий уровень)
class DatabaseLayer {
    public String fetchData(int id) throws Exception {
        if (id <= 0) {
            // Симулируем SQL ошибку
            throw new Exception("SQL Error: Invalid ID " + id);
        }
        return "данные_" + id;
    }
}

// Уровень 2: Сервисный слой
class DataService {
    private DatabaseLayer db = new DatabaseLayer();

    public String getData(int id) throws ServiceException {
        try {
            return db.fetchData(id);
        } catch (Exception e) {
            throw new ServiceException("Не удалось получить данные для id=" + id, e);
        }
    }
}

// Пользовательское исключение сервисного слоя
class ServiceException extends Exception {
    public ServiceException(String message, Throwable cause) {
        super(message, cause);
    }
}

// Уровень 3: Слой приложения
public class Application {
    public static void main(String[] args) {
        DataService service = new DataService();

        // Тест с корректным id
        try {
            System.out.println(service.getData(1));
        } catch (ServiceException e) {
            System.out.println("Ошибка: " + e.getMessage());
        }

        // Тест с некорректным id
        try {
            System.out.println(service.getData(-1));
        } catch (ServiceException e) {
            System.out.println("Ошибка сервиса: " + e.getMessage());
            System.out.println("Причина: " + e.getCause().getMessage()); // Исходное исключение
            System.out.println("\nПолный стек:");
            e.printStackTrace();
        }
    }
}
```

Часть 5: Самостоятельная работа

Задание 5.1: Обобщённый кэш

Реализуйте обобщённый LRU-кэш `SimpleCache<K, V>` с ограниченным размером. Используйте `LinkedHashMap` с `accessOrder=true` и переопределённым `removeEldestEntry`. Методы: `put(K key, V value)`, `get(K key)`, `containsKey(K key)`, `size()`, `clear()`.

Продемонстрируйте: создайте кэш ёмкостью 3, добавьте `a=1, b=2, c=3`. Добавьте `d=4` — должен вытеснить наименее используемый. Проверьте, что `get("a")` вернёт `null`.

Задание 5.2: Обобщённый Result тип

Реализуйте обобщённый тип `Result<T>` — обёртку для результата операции:

- Фабричные методы: `static <T> Result<T> success(T value)` и `static <T> Result<T> failure(Exception error)`.
- `isSuccess()`, `getValue()`, `getError()`.
- `getOrDefault(T defaultValue)` — возвращает значение при успехе, иначе `defaultValue`.
- `<R> Result<R> map(Function<T, R> mapper)` — трансформирует значение при успехе; при неуспехе возвращает `Result.failure` с той же ошибкой.
- `toString()`.

Реализуйте `static Result<Integer> divide(int a, int b)`, продемонстрируйте `divide(10,2)` и `divide(10,0)`, `getOrDefault`, цепочку `map`.

Часть 6: Контрольные вопросы

Ответьте письменно:

1. В чём разница между нестатическим внутренним классом и статическим вложенным классом?
2. Почему для создания экземпляра нестатического внутреннего класса снаружи нужен экземпляр внешнего класса?
3. Что такое стирание типов (type erasure)? Приведите пример того, что нельзя сделать из-за этого.
4. Объясните принцип PECS. Когда использовать `? extends T` и когда `? super T`?
5. Чем отличается `throws` от `throw`?
6. В каких случаях нужно создавать собственные checked исключения, а в каких — unchecked?
7. Что произойдёт, если исключение возникнет внутри блока `finally`?
8. Какой интерфейс должен реализовывать класс для использования в try-with-resources?
9. Для чего используется цепочка исключений (exception chaining)?
10. Можно ли создать массив обобщённого типа (`new T[10]`)? Объясните почему.

Результаты занятия

К концу занятия вы должны сдать:

1. Реализованные Java-файлы для всех заданий
2. Ответы на контрольные вопросы
3. Краткое описание того, какие задачи вызвали затруднения

Критерии оценки:

- Все программы компилируются без ошибок
- Правильные результаты выполнения
- Корректное использование обобщений (без `@SuppressWarnings` без необходимости)
- Осмысленная обработка исключений (не пустые `catch` блоки)
- Соблюдение принципов ООП